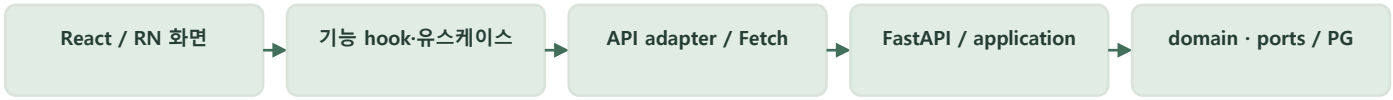
01 / IMPLEMENTATION NOTES

무엇을 만들었고, 어떻게 연결했나

P1 네 기능을 기존 P0의 인증·AI 작업·기록·개인정보 수명주기에 연결했다. 이 보고서는 현재 코드와 이전 실행 로그를 대조한 구현 해설이다. 이번 문서 작업에서 제품 코드와 테스트는 변경하지 않았다.

기능	사용자 동작	구현의 중심
통계·달력	7/30일 확인 → 날짜별 기록 열기	SQL 집계, 저장된 diary day 기준, AI 호출 없음
즐겨찾기	v2 리딩 저장 → 재열기 → 해제	owner+reading 복합 FK, 멱등 요청, 원본 삭제 cascade
선택 기록 요약	1~10개 선택 → 작업 → 관찰·조언·행동	선택 ID/version, 목적별 동의, PG job/lease, 결과 무효화
기기 알림	다이얼 → 명시적 켜기 → OS 권한 → 끄기	JS 경쟁 제어 + Android 로컬 alarm, 웹 미지원 안내



저장소 분담과 읽는 순서

저장소	책임	분석 기준 커밋
taro	RN Community CLI, native bridge, Android·OTA 도구	7aedace04712
taro-raect	React Native Web 화면, 브라우저 빌드·통합 테스트	5339b8b45b04
taro-back	FastAPI application/domain/adapter, PostgreSQL, worker	d50e221291a1

가장 중요한 코드는 P1Api → AIJobsService.create_insight → run_next → JournalInsightsService._valid 순서다. 선택한 서버 원본과 현재 동의를 확인한 뒤 작업을 생성하고, 호출 전후에도 원본과 동의를 재검증한다.

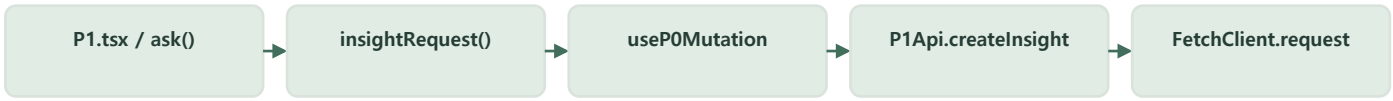
스택: React 19.2.3 / RN 0.85.3 / Node 24.22.2(빌드·OTA) / Python FastAPI / PostgreSQL 17. 앱 1.3.0, Android code 4. 실제 OpenAI/전사 호출과 운영 배포는 검증 범위 밖이다.

PDF 책갈피로 각 주제로 이동할 수 있다. 코드는 함께 제공한 Markdown 원문에서 복사할 수 있으며, 회색 줄번호는 보고서 내부 표시 번호다.

02 / IMPLEMENTATION NOTES

프론트: 화면과 HTTP를 분리한 위치

화면은 입력·선택·이동을 담당한다. 요청 조립·응답 검증은 P1Api, 인증 헤더·timeout·HTTP 오류는 FetchClient가 담당한다. composition.ts에서 같은 JsonTransport를 주입한다.



taro / src/features/p1/P1Api.ts:37-37 · 발췌 · 표시 형식 정리

```
async createInsight(input: InsightInput, key: string) {
  const checked = validateInsightInput(input);
  id(key);
  return accepted(
    await this.http.request('/v2/journal-insights', {
      method: 'POST',
      headers: { 'X-Idempotency-Key': key },
      body: JSON.stringify(checked)
    })
  );
}
```

P1Api는 fetch 전역을 직접 호출하지 않는다. JsonTransport.request를 받으므로 테스트에서는 작은 transport fake를 넣어 경로·키·본문·잘못된 응답을 검사한다. accepted()는 job_id/resource_id/state/status_url/events_url을 디코딩한다.

taro / src/infrastructure/http/FetchClient.ts:22-27 · 발췌 · 표시 형식 정리

```
const token = authenticated ? await this.getToken() : null;
const headers = new Headers(options.headers);
if (!(typeof FormData !== 'undefined' && options.body instanceof FormData))
  headers.set('Content-Type', 'application/json');
if (token) headers.set('Authorization', `Bearer ${token}`);
const response = await this.fetcher(this.baseUrl + path, {
  ...options,
  headers,
  signal: controller.signal
});
const result: unknown = response.status === 204 ? null : await response.json();
```

서버 snake_case 계약은 API adapter에서 타입과 의미를 확인한다. 예를 들어 집계는 기분 합계가 전체 횟수와 맞는지, 요약은 ready일 때만 insight 결과가 존재하는지까지 검사한다. 실패 응답을 성공 화면으로 바꾸지 않는다.

useP0Mutation은 기존 operation receipt·재시도 키 정책을 재사용한다. 화면에서 mutation을 만든 이름은 P1이지만 hook 이름에는 기존 P0가 남아 있다. P1Api가 P0Api의 decoder를 import하는 결합도 현재 코드에 존재한다.

03 / IMPLEMENTATION NOTES

프론트: 느린 응답과 선택 버전 처리

7일 요청 후 30일로 바꿨을 때 먼저 보낸 응답이 늦게 도착할 수 있다. 기록 선택에서는 사용자가 읽은 version을 보관해야 이후 수정된 원문이 조용히 전송되지 않는다.

taro / src/features/p1/SummaryLoader.ts:9-13 · 발췌 · 표시 형식 정리

```
async load(from: string, to: string) {
  const generation = ++this.generation;
  this.value = { value: null, busy: true, error: '' };
  try {
    const next = await this.api.summary(from, to);
    if (generation === this.generation)
      this.value = { value: next, busy: false, error: '' };
  } catch (e) {
    if (generation === this.generation)
      this.value = {
        value: null,
        busy: false,
        error: e instanceof Error ? e.message : '기록을 확인하지 못했어요.'
      };
  }
}
```

generation이 현재 요청과 같을 때만 화면 상태를 갱신한다. dispose()도 세대를 올린다. 이는 늦은 응답의 반영을 막는 방식이며, 이미 나간 HTTP 요청을 자동 취소하거나 서버 집계 비용을 없애는 방식은 아니다.

taro / src/features/p1/selection.ts:19-23 · 발췌 · 표시 형식 정리

```
export function insightRequest(
  selection: InsightSelection,
  bootstrap: Pick<Bootstrap, 'consents' | 'policy_version'>
): InsightInput {
  const checked = insightSelection(
      selection.range_start,
      selection.range_end,
      selection.entries
    ),
    ai = currentConsent(bootstrap.consents, 'ai_processing', bootstrap.policy_version),
    context = currentConsent(bootstrap.consents, 'journal_context', bootstrap.policy_version);
  if (!ai || !context)
    throw new Error('AI 처리와 마음기록 참조를 각각 허용한 뒤 다시 요청해 주세요.');
  return validateInsightInput({
    ...checked,
    entry_ids: checked.entries.map((e) => e.id),
    entry_versions: Object.fromEntries(checked.entries.map((e) => [e.id, e.version])),
    consent_event_id: ai.id
  });
}
```

요약 선택 draft에는 ID·version·local_day와 범위만 보관한다. 본문은 선택 draft에 복제하지 않는다. 실제 요청 직전 bootstrap을 다시 읽어 최신 AI 처리·기록 참조 동의로 input을 만든다. 기존 타로/대화 최대 3개 문맥과 요약 최대 10개 선택은 별도 저장 키다.

04 / IMPLEMENTATION NOTES

백엔드: Hexagonal 경계와 UoW

FastAPI router는 HTTP 입출력을, application은 트랜잭션과 유스케이스 순서를, domain은 규칙을 담당한다. SQL과 psycopg 의존성은 PostgreSQL adapter에 둔다. 생성자와 OpenAPI 생성에서 DB를 열지 않는다.

taro-back / api/main.py:109-119 · Composition root: 서비스와 기존 수명주기 연결

```
journals_v2 = JournalV2UseCases(lambda: PostgresJournalV2UnitOfWork(pool), journal_v2_policy,
    source_exists=references.exists, invalidate_context=ai_jobs.cancel_for_journal)
summary = JournalSummaryUseCases(lambda: PostgresJournalSummaryUnitOfWork(pool), journal_v2_policy)
favorites = FavoritesUseCases(lambda: PostgresFavoritesUnitOfWork(pool))
insights = JournalInsightsService(lambda: PostgresJournalInsightsUnitOfWork(pool), ai_jobs=ai_jobs,
    owner_guard=owner_guard, policy=ai_policy)
ai_jobs.insight_source = insights
data.export_callbacks = (ai_jobs.export_owner, readings.export_owner, journals_v2.export_owner,
                         favorites.export_owner, insights.export_owner)
data.erase_callbacks = (readings.erase_owner, ai_jobs.erase_owner, journals_v2.erase_owner,
                        favorites.erase_owner, insights.erase_owner)
```

여기서 concrete adapter를 골라 주입한다. journal 편집/삭제 hook과 AI job의 source port가 연결되고, export/erase에 새 데이터셋을 등록한다. insights↔ai_jobs는 setter를 포함한 상호 참조가 남아 있으며 마지막 평가에서 다룬다.

taro-back / api/application/journal_summary.py:14-19

```
def summary(self, owner, *, start, end):
    validate_summary_range(start, end)
    with self.uow_factory() as uow:
        uow.summary.lock(owner)
        uow.summary.active(owner)
        return journal_summary(start, end, uow.summary.groups(owner, start, end), self.policy)
```

taro-back / api/adapters/journal_summary_postgres.py:17-21

```
def groups(self, owner, start, end):
    rows = self.connection.execute('''SELECT day AS local_day,mood,count(*) AS entry_count
        FROM journal_entries WHERE session_id=%s AND day>=%s AND day<=%s
        GROUP BY day,mood ORDER BY day,mood''', (owner, start, end)).fetchall()
    return [JournalMoodBucket(row['local_day'], row['mood'], row['entry_count']) for row in rows]
```

집계는 body를 SELECT하지 않는다. journal_entries의 저장된 day를 양끝 포함 범위로 묶고, 기존 JournalV2Policy가 한국어/영문 mood 매핑을 정규화한다. 조회 때 별도 통계를 저장하지 않으므로 새 export/erase 데이터셋도 만들지 않았다.

05 / IMPLEMENTATION NOTES

즐겨찾기: 앱 검사와 DB 제약을 겹친 이유

reading_id만 참조하면 다른 소유자의 리딩을 연결하는 adapter 버그를 DB가 막지 못한다. composite FK로 owner와 reading의 일치를 강제하고, 삭제는 원본 테이블의 cascade에 맡겼다.

taro-back / api/migrations/0014_p1_journal_data.sql:4-10

```
CREATE TABLE IF NOT EXISTS favorites (
    owner_id uuid NOT NULL REFERENCES app_sessions(id) ON DELETE CASCADE,
    reading_id uuid NOT NULL,
    created_at timestamptz NOT NULL DEFAULT clock_timestamp(),
    PRIMARY KEY(owner_id,reading_id),
    FOREIGN KEY(owner_id,reading_id) REFERENCES p0_readings(owner_id,id) ON DELETE CASCADE
);
```

taro-back / api/application/favorites.py:19-37

```
def add(self, owner, values, request_key):
    reading_id = favorite_input(values)
    key_hash, fingerprint = favorite_hash(favorite_uuid(request_key)), favorite_hash(reading_id)
    with self.uow_factory() as uow:
        repository = uow.favorites
        self._owner(repository, owner)
        previous = repository.request(owner, key_hash)
        if previous is not None:
            if previous.fingerprint != fingerprint:
                raise FavoriteConflict('idempotency_conflict')
            if previous.reading_id is None or repository.get(owner, previous.reading_id) is None:
                raise FavoriteNotFound('favorite_reference_deleted')
            return FavoriteRecord(previous.reading_id, previous.created_at).dto()
        if not repository.has_reading(owner, reading_id):
            raise FavoriteNotFound('favorite_reading_not_found')
        saved = repository.get(owner, reading_id) or repository.add(owner, reading_id)
        repository.remember(owner, key_hash, fingerprint, saved)
        uow.commit()
        return saved.dto()
```

같은 키·같은 리딩은 원래 created_at을 반환한다. 같은 키·다른 리딩은 409다. 즐겨찾기나 원본을 지운 뒤 옛 요청을 replay해도 새 행을 만들지 않는다. 원본을 다시 저장하려면 새 키를 쓴다. 이미 명시적으로 재저장한 뒤 옛 키를 replay하면 원래 요청의 timestamp가 반환될 수 있다.

원본 삭제 후 key metadata의 reading 참조는 SET NULL로 남아 요청이 소비됐음을 기억한다. metadata에는 키 hash·fingerprint·참조·timestamp만 있으며 owner 전체 삭제 때 지운다. 기본 페이지 20 / 최대 50, (created_at, reading_id) 내림차순 keyset paging이다.

06 / IMPLEMENTATION NOTES

선택 기록 요약: 202로 접수하는 계약

한 HTTP 요청에서 모델 응답까지 기다리지 않는다. 선택 메타데이터, 실행 작업, 한도 예약을 한 트랜잭션에서 커밋한 뒤 JobAccepted를 반환한다. 원문은 요청 body에 포함하지 않는다.

계약 설명용 요청: UUID는 예시이며 owner는 인증 세션에서 결정

```
POST /v2/journal-insights
Authorization: Bearer <session-token>
Content-Type: application/json
X-Idempotency-Key: 00000000-0000-4000-8000-000000000003

{
  "range_start": "2026-11-01", "range_end": "2026-11-06",
  "entry_ids": ["00000000-0000-4000-8000-000000000001"],
  "entry_versions": {"00000000-0000-4000-8000-000000000001": 2},
  "consent_event_id": "00000000-0000-4000-8000-000000000002"
}
```

taro-back / api/application/ai_jobs.py:321-345

```
def create_insight(self, owner, key, body):
    body = insight_input(body, self.policy['insight_entries_max'])
    if self.insight_source is None:
        raise AIError('AI_UNAVAILABLE', 503)
    with self._uow() as uow:
        c = uow.ai_jobs
        self._owner(c, owner)
        previous = self._once(c, owner, 'insight', key, body)
        if previous:
            return previous
        sources = self._sources(c, owner, body['entry_ids'], body.get('entry_versions'),
                                maximum=self.policy['insight_entries_max'])
        if any(not body['range_start'] <= str(source['day']) <= body['range_end'] for source in sources):
            raise AIError('INVALID_INSIGHT_RANGE')
        resource = uuid4()
        job = self._new_job(c, owner, 'insight', resource, 1, body['consent_event_id'], body['entry_ids'], {},
                            context_entry_versions=body.get('entry_versions'))
        self.insight_source.create_source(c.transaction, owner, resource, body, sources,
                                          job['consent_event_id'], job['context_consent_id'])
        if not self.insight_source.mark_generating(c.transaction, owner, resource, 1):
            raise AIError('RESOURCE_CHANGED', 409)
        result = c.accepted(job)
        self._remember(c, owner, 'insight', key, body, resource, result)
        uow.commit()
        return result
```

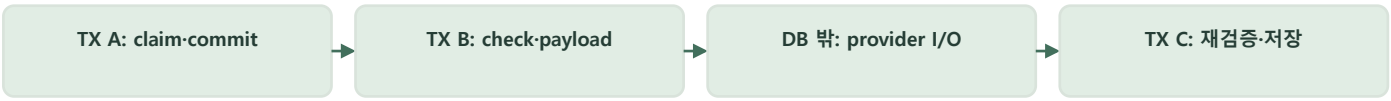
202 응답은 job_id·resource_id·state·status_url·events_url이다. resource_id가 insight ID이며 완료 후 GET /v2/journal-insights/{id}로 결과를 연다. entry_versions는 호환성을 위해 선택적 입력이지만 신규 화면은 항상 보낸다. 제공한 version map은 선택 ID 전체와 정확히 일치해야 한다.

동의 없음 403 / 타인·없는 원본 404 / 수정된 원본 409 / 한도·동시 실행 429 / 제공자 미준비 503을 구분한다. 인가·범위·버전 검증에 실패한 요청은 새 작업·한도 예약을 남기지 않도록 검사했다.

07 / IMPLEMENTATION NOTES

Worker: 외부 I/O 동안 DB 연결을 놓는다

PG의 queued 작업을 SKIP LOCKED로 claim하고 lease_id를 부여한다. 원본 확인과 결과 저장은 짧은 별도 UoW에서 수행하며 제공자 호출 동안 DB 연결·owner lock을 유지하지 않는다.



taro-back / api/application/ai_jobs.py:534-541 · TX A: 실행권을 먼저 확정

```
def run_next(self, worker_id='worker'):
    lease = uuid4()
    with self._uow() as uow:
        c = uow.ai_jobs
        row = c.claim_next(lease, self.policy['lease_seconds'])
        if not row:
            return False
        uow.commit()
```

taro-back / api/application/ai_jobs.py:559-565 · TX B 끝: provider_started를 커밋하고 UoW 종료

```
    c.mark_provider_started(job['id'], lease, job['kind'] != 'chat' or job['payload'].get('use_ai', True))
    if job['kind'] == 'chat':
        c.set_turn_state(job['owner_id'], job['resource_id'], 'generating')
    elif job['kind'] == 'transcription':
        c.set_transcription_state(job['resource_id'], 'running')
    uow.commit()
# No open unit of work, connection, transaction or owner lock across provider I/O.
```

taro-back / api/application/ai_jobs.py:582-588 · DB 밖: 실제 호출과 출력 검증

```
        else:
            output = self.provider.generate(payload)
        output = validate_result(output, job['kind'], payload['cards'])
except AIError as failure:
    error = failure.code
except Exception:
    error = 'UNKNOWN_PROVIDER_OUTCOME'
```

taro-back / api/application/ai_jobs.py:589-602 · TX C: 실행권·취소·원본·동의 재검증

```
with self._uow() as uow:
    c = uow.ai_jobs
    c.lock_owner(row['owner_id'])
    current = c.job(row['owner_id'], row['id'], for_update=True)
    if not current or current['lease_id'] != lease or current['state'] in TERMINAL:
        return True
    try:
        if current['state'] != 'running':
            raise AIError('CANCELLED', 409)
        self._check(c, current)
    except AIError as failure:
        self._finish(c, current, 'cancelled', failure.code)
    else:
        self._finish(c, current, 'failed' if error else 'succeeded', error, output)
```

프로세스가 호출 시작 후 죽으면 외부 요청이 성공했는지 알 수 없다. recover()는 provider_started=true의 만료 작업을 UNKNOWN_PROVIDER_OUTCOME 실패로 두며 자동 재호출하지 않는다. 사용자의 명시적 retry만 새 작업과 예약을 만든다. 이미 외부로 보낸 요청의 철회·원격 취소는 보장하지 않고 늦은 결과 저장을 막는다.

08 / IMPLEMENTATION NOTES

원본 수정·삭제·동의 철회 후 결과 수명

AI 결과는 원본의 고정된 해석이 아니라 특정 원본 version과 동의 이벤트에 매인 파생 데이터다. 접수·payload 조립·결과 저장·결과 조회에서 이를 다시 확인한다.

taro-back / api/application/journal_insights.py:33-48

```
def _valid(self, repository, row):
    sources = row['sources']
    if (row['state'] == 'invalidated' or not 1 <= len(sources) <= self.policy['insight_entries_max']
            or row['based_on_count'] != len(sources)):
        return False
    expected = {str(source['entry_id']): source['entry_version'] for source in sources}
    current = repository.current_sources(row['owner_id'], [source['entry_id'] for source in sources])
    if len(current) != len(sources) or any(expected.get(str(source['id'])) != source['version']
            or not str(row['range_start']) <= str(source['day']) <= str(row['range_end']) for source in curren
    t):
        return False
    for kind, field in (('ai_processing', 'consent_event_id'), ('journal_context', 'context_consent_id')):
        latest = repository.latest_consent(row['owner_id'], kind)
        if (not latest or not latest['granted'] or latest['policy_version'] != self.policy['policy_version']
                or str(latest['id']) != str(row[field])):
            return False
    return True
```

taro-back / api/application/ai_jobs.py:652-658

```
def cancel_for_journal(self, connection, owner, entry_id):
    repository = self._repository(connection)
    repository.lock_owner(owner)
    for job in repository.pending_for_journal(owner, entry_id):
        self._cancel(repository, job)
    if self.insight_source:
        self.insight_source.invalidate_for_journal(repository.transaction, owner, entry_id)
```

taro-back / api/adapters/journal_insights_postgres.py:59-61

```
def invalidate(self, owner, insight_id):
    self.transaction.execute("""UPDATE journal_insights SET state='invalidated',result=NULL,version=version+1
        WHERE owner_id=%s AND id=%s AND state!='invalidated'""", (owner, insight_id))
```

선택 개수·원본 생존·정확한 version·범위·목적별 최신 true 동의 이벤트 ID가 같아야 유효하다. 재동의해도 과거 결과는 되살아나지 않는다. journal_insight_sources는 원문 없이 ID/version만 유지하며 원본 journal과 강제 FK를 두지 않았다. owner/insight 삭제 때는 cascade로 제거한다.

GET도 현재 유효성을 확인한 후 필요하면 invalidated/result=NULL UPDATE를 커밋하는 읽기 시 정리 경로다. 순수 SELECT endpoint는 아니다. 변경 hook과 조회 시 확인을 함께 두어 hook을 거치지 않은 변경에도 오래된 결과를 표시하지 않게 했다.

09 / IMPLEMENTATION NOTES

전체 삭제: 서버와 기기 정리를 잇는 위치

서버 전체 삭제가 completed로 확인되기 전에는 로컬 연결을 지우지 않는다. 완료 영수증은 남겨 재조회할 수 있게 하고, 기기의 draft·작업 선택·알림·연결 정보는 해당 단계에서 정리한다.

taro-back / api/main.py:116-119 · 새 데이터셋을 기존 export/erase 트랜잭션에 등록

```
data.export_callbacks = (ai_jobs.export_owner, readings.export_owner, journals_v2.export_owner,
                         favorites.export_owner, insights.export_owner)
data.erase_callbacks = (readings.erase_owner, ai_jobs.erase_owner, journals_v2.erase_owner,
                        favorites.erase_owner, insights.erase_owner)
```

taro-back / api/application/favorites.py:64-66

```
def erase_owner(self, connection, owner):
    # Privacy orchestration owns this transaction and has already locked the owner.
    self.uow_factory().repository_for(connection).erase_owner(owner)
```

taro / src/features/p0/eraseCurrentDeviceData.ts:4-14 · 발췌 · 표시 형식 정리

```
export async function eraseCurrentDeviceData(
  storage: OwnedStorage,
  identity: { forget(): Promise<void> },
  receipt: DataReceipt,
  clearDeviceReminders?: () => Promise<unknown>
): Promise<void> {
  if (receipt.kind !== 'erase' || receipt.state !== 'completed')
    throw new Error('서버의 전체 삭제 완료를 먼저 확인해 주세요.');
  if (receipt.device_cleanup_complete === true) return;
  const receiptKey = 'yeoun.p0.data-receipt';
  await storage.setItem(receiptKey, JSON.stringify(receipt));
  await clearDeviceReminders?.();
  const keys = await storage.getAllKeys();
  await storage.multiRemove(
    keys.filter((key) => key.startsWith('yeoun.') && key !== receiptKey)
  );
  await identity.forget();
  await storage.setItem(
    receiptKey,
    JSON.stringify({ ...receipt, device_cleanup_complete: true })
  );
}
```

clearDeviceReminders는 선택적 포트 callback으로 추가했다. Store에서는 reminderController.reset()을 넘긴다. 이미 device_cleanup_complete=true인 영수증은 즉시 종료하므로 과거 삭제 화면을 다시 열어 새 연결의 설정을 지우는 일을 방지한다.

서버 export/erase callback은 caller의 transaction을 재사용한다. 요약 export는 현재 유효성을 재검증하고 journal_insights dataset을, 즐겨찾기는 v2_favorites dataset을 반환한다. 전체 삭제 중 provider가 돌아와도 재검증에서 owner/resource가 없어 결과를 복원하지 못한다. 익명화된 일일 과금 집계는 삭제로 한도를 초기화하지 않도록 유지한다.

브라우저 실제 API 통합에서는 ZIP·전체 삭제·연결 401·영수증 reload까지 확인했다. 네이티브 알림 clear bridge는 빌드와 공유 포트 경쟁 테스트로 확인했으며, 기기에서 서버 삭제부터 OS 정리까지 전 과정을 측정했다고 표시하지 않는다.

10 / IMPLEMENTATION NOTES

알림 JS: 늦은 권한 허용이 OFF를 뒤집지 않게

권한 팝업을 기다리는 동안 사용자가 끄거나 전체 삭제할 수 있다. generation으로 오래된 의도를 버리고 serial queue로 네이티브 저장·끄기·clear 순서를 제어한다.

taro / src/features/reminders/ReminderController.ts:10-10 · 발췌 · 표시 형식 정리

```
private serial<T>(operation: () => Promise<T>): Promise<T> {
  const result = this.tail.then(operation, operation);
  this.tail = result.catch(() => {});
  return result;
}
```

taro / src/features/reminders/ReminderController.ts:12-23 · 발췌 · 표시 형식 정리

```
async enable(time: ReminderTime): Promise<ReminderState> {
  validateReminderTime(time);
  const generation = ++this.generation;
  const before = await this.read();
  if (!before.supported || generation !== this.generation) return this.read();
  if (!this.permission) {
    const pending = this.port.requestPermission();
    this.permission = pending;
    void pending
      .finally(() => {
        if (this.permission === pending) this.permission = undefined;
      })
      .catch(() => {});
  }
  const permission = await this.permission;
  return this.serial(async () => {
    if (generation !== this.generation || permission !== 'granted') return this.port.read();
    const state = await this.port.schedule(time);
    if (generation !== this.generation) return this.port.disable();
    return state;
  });
}
```

disable()/reset()은 generation을 먼저 올리고 각각 disable/clear를 같은 queue에 넣는다. schedule 자체가 늦게 끝나도 세대가 바뀌었으면 저장을 취소한다. 동시 enable의 권한 요청은 공유하고 마지막 시간 선택만 유지한다.

화면은 ReminderPort만 사용한다. native adapter는 Android 모듈/POST_NOTIFICATIONS에 연결하고, web adapter와 이전 네이티브 앱은 supported=false로 반환한다. 화면 진입만으로 권한을 요청하지 않는다. 사용자가 직접 켰던 예약의 OS 재진입 복구와 최초 자동 활성화는 구분한다.

11 / IMPLEMENTATION NOTES

알림 Native: 매일 24시간을 더하지 않는다

기기 시간대의 달력 날짜를 이동시켜 다음 예정 시각을 구한다. 서머타임 전환일은 하루가 24시간과 다를 수 있다. 정확한 시각을 보장하는 알람 권한은 사용하지 않았다.

taro / android/app/src/main/java/com/yeoun/app/YeounReminderScheduler.kt:41-46

```
private fun arm(context: Context, hour: Int, minute: Int): Long {
  val next = ReminderClock.next(System.currentTimeMillis(), TimeZone.getDefault(), hour, minute)
  // Daily reflection does not require exact-alarm access or a continuously running service.
  context.getSystemService(AlarmManager::class.java).setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, next, pendi
    ng(context))
  return next
}
```

taro / android/app/src/main/java/com/yeoun/app/ReminderClock.java:9-27

```
public static long next(long now, TimeZone zone, int hour, int minute) {
  if (hour < 0 || hour > 23 || minute < 0 || minute > 59) throw new IllegalArgumentException("Invalid reminder
     time");
  Calendar target = Calendar.getInstance(zone);
  target.setTimeInMillis(now);
  target.set(Calendar.HOUR_OF_DAY, hour);
  target.set(Calendar.MINUTE, minute);
  target.set(Calendar.SECOND, 0);
  target.set(Calendar.MILLISECOND, 0);
  if (target.getTimeInMillis() <= now) {
    // Add the date before setting a potentially nonexistent spring-DST wall time.
    target.setTimeInMillis(now);
    target.add(Calendar.DAY_OF_YEAR, 1);
    target.set(Calendar.HOUR_OF_DAY, hour);
    target.set(Calendar.MINUTE, minute);
    target.set(Calendar.SECOND, 0);
    target.set(Calendar.MILLISECOND, 0);
  }
  return target.getTimeInMillis();
}
```

같은 immutable PendingIntent 하나를 교체하므로 시간 변경 후 예약이 늘지 않는다. SharedPreferences에는 enabled/hour/minute/next_at만 저장한다. 권한·채널이 차단되면 예약을 취소하고 꺼짐으로 표시한다. 알림 본문은 공통 문구이며 개인 일기는 넣지 않는다.

재부팅·시각/시간대 변경·패키지 업데이트 receiver는 이전에 켠 설정만 복구한다. API 35 별도 테스트 앱에서 20:00→21:00 변경 후 예약 1개, 시스템 미리 보기, 끄기 후 0개와 재진입을 확인했다. 실제 다음 날 배달과 제조사별 절전 동작은 측정하지 않았다.

코드푸시 경계: JS·화면·정적 에셋 업데이트는 기존 OTA 경로를 사용한다. 새 Kotlin/Java 모듈·Manifest 권한은 새 기본 앱이 필요해 1.3.0/code 4로 올렸다. 기존 기본 앱에는 알림 미지원 안내가 표시된다. 이번 P1 운영 OTA를 게시했다는 의미는 아니다.

12 / IMPLEMENTATION NOTES

TDD: 실제로 실패시킨 동작 한 가지

모듈이 없어서 collection이 실패한 최초 RED만으로 설계가 검증됐다고 보지 않는다. 아래는 첫 원본 version 검사 직후 데이터가 바뀌었을 때, 새 본문이 제공자에게 넘어가던 문제를 재현한 행동 검증이다.

taro-back / api/tests/unit/test_p1_insights_ports.py:200-219

```
def test_worker_does_not_send_a_new_source_version_changed_after_initial_check(monkeypatch):
    state = State()
    original_version = JobsRepository.journal_version
    def version_then_edit(repository, owner, entry):
        version = original_version(repository, owner, entry)
        state.journals[state.entry].update(version=2, body='unapproved newer text')
        return version
    monkeypatch.setattr(JobsRepository, 'journal_version', version_then_edit)
    calls = []
    class Provider:
        def available(self): return True
        def generate(self, payload):
            calls.append(payload)
            return dict(RESULT)
    factory = lambda: UnitOfWork(state)
    source = JournalInsightsService(factory)
    service = AIJobsService(factory, provider=Provider(), insight_source=source)
    assert service.run_next()
    assert calls == []
    assert state.job['state'] == 'cancelled' and state.row['state'] == 'invalidated'
```

보관된 RED 로그의 핵심 결과 · 06-preprovider-red.txt

```
assert calls == []
E AssertionError: assert [{'kind': 'insight', ...}] == []
1 failed, 8 passed
```

taro-back / api/application/ai_jobs.py:193-201 · payload 재구성에 사용한 _sources의 exact version 검사

```
expected_versions = validate_context_entry_versions(ids, versions)
rows = []
for entry in ids:
    row = c.journal(owner, entry)
    if not row:
        raise AIError('NOT_FOUND', 404)
    if expected_versions is not None and expected_versions[str(row['id'])] != row['version']:
        raise AIError('RESOURCE_CHANGED', 409,
                      '선택한 기록이 바뀌었어요. 내용을 확인한 뒤 다시 선택해 주세요.')
```

_payload()가 원문을 다시 읽을 때도 context_versions를 넘기도록 보강했다. 접수 또는 초기 검사만 통과했다고 새 원문을 허용하지 않는다. 같은 테스트를 약화하지 않고 provider 호출 0회·job cancelled·insight invalidated를 확인했다. 과거 RED 로그를 검토한 것이며 이번 보고서 작업에서 제품 테스트를 새로 실행해 실패시킨 것은 아니다.

13 / IMPLEMENTATION NOTES

검증과 Git: 증거가 뜻하는 범위

숫자는 테스트 케이스 수이며 성능·커버리지·운영 안정성 점수가 아니다. 아래는 이전 구현 작업에서 보관한 실행 로그와 커밋을 이번에 읽기 전용으로 대조한 결과다.

검증	기록된 결과	실제로 확인한 범위
FastAPI / PG	390 passed	기존 226 + 새 164. 단위·소유권·실DB 제약·경쟁·삭제 수명주기
RN / Web unit	71 / 68 passed	decoder·기간 응답 경쟁·선택 버전·알림/삭제 순서
Browser	21 fixture + 4 real API	모의 UI 계약 + 실PG/worker/ZIP/전체 삭제. 모델만 fake
JVM / Android	6 / build PASS	시간대·같은 분·연말·DST·잘못된 시간. API35 동작 검사 별도
Type / Web build	PASS	두 타입 검사, production build. JS chunk 경고 약679kB/gzip190kB

GREEN 단위로 남긴 주요 커밋

저장소	커밋	검토 단위
backend	fb9b5d4 / 74e88cc	통계·달력 집계 / 즐겨찾기·DB 제약
backend	0e39f21 / b450a62 / 9b4de93	요약 domain/storage → jobs·HTTP·수명주기 → 점수 확정 출력 검사
RN / Web	240db1a / 651d5ed	P1 화면·typed API·선택·달력·작업 완료 분기
RN	08b57c5 / 66cbc27	알림 포트·경쟁 제어 → native bridge·JVM·1.3.0
backend / Web	f64a538 / 994c25d	composition·OpenAPI → 실제 API 브라우저 통합

RED 확인 → 구현·리팩터링 → GREEN → 기능 단위 commit/push 순서로 작업했다. 테스트와 구현은 같은 GREEN 커밋에 담겨 있으며, git log만으로 모든 중간 RED 실행 순서를 재생할 수는 없다. RED/GREEN 로그와 실제 assertion을 함께 근거로 삼는다. 기존 테스트 파일 변경·삭제·완화·skip은 없었다.

세 PR #2는 codex/complete-core-flows-20261106을 base로 한 후속 draft다. 원격 SHA 일치까지 확인했고 main 병합·운영 배포는 수행하지 않았다. 재현 명령과 fixture는 기존 docs/quality/p1-reproduce.md 및 api/tests/support/preview_api.py에 보관했다. 이번 문서 추가로 기능 테스트를 반복 실행하지 않았다.

14 / IMPLEMENTATION NOTES

시니어 관점 평가와 다음 작업 순서

현재 구현은 ports/adapters·UoW를 중심으로 기능을 분리했고, 데이터 보전과 비동기 경쟁의 검증 근거가 있다. 운영 확장성과 도메인 경계에는 아래 정리할 부분이 남아 있다. 이 표는 평가이며 이번에 코드를 바꾸지는 않았다.

현재 코드에서 확인한 점	영향	다음 개선안
RN/Web의 P1Api·화면·controller를 각 repo에 복사 관리	현재 주요 5개 파일이 byte 동일하지만 이후 동기화 누락 위험	공통 package 또는 계약 생성 범위를 먼저 확정
P1Api가 P0Api decoder를 import, P1 화면에서 useP0Mutation 재사용	단계 이름이 기술 공통 모듈 역할까지 겸함	공통 HTTP decoder·mutation을 명시 모듈로 분리
AIJobsService↔JournalInsightsService setter 상호 참조, dict 기반 상태	조립 순서·콜백 시그니처 의존, 정적 검증 한계	명확한 source/result port·타입 DTO와 callback 계약 정리
domain.ai_jobs.load_policy()가 JSON 파일을 직접 읽음	domain에 설정 I/O가 남음. DB-free가 I/O-free를 뜻하지 않음	composition에서 읽은 불변 policy를 주입
10·20·50 상한이 클라/HTTP/domain에 반복됨	계약 상한·정책값 변경 시 수동 동기화 필요	계약 상한과 운영 설정을 구분하고 단일 source/생성 검토
isFavorite() 전체 페이지 탐색, export all_owned의 행별 source 조회	기록이 많으면 O(N) 탐색·N+1 비용이 커질 수 있음	실측 후 조회 API·batch read·index/query plan 검토
FastAPI process 내부 worker loop, PG durable jobs	작업 상태는 DB에 남으나 배포·종료·처리량 운영 설계 필요	동일 usecase를 실행하는 별도 worker 역할·관측·배포 절차
favorites 소비 키 metadata는 owner 삭제까지 보관	요청 이력 증가와 보관 정책을 운영에서 다뤄야 함	멱등 보장 기간을 계약으로 정한 뒤 purge 정책 검토

다음 구현 전에 확정할 것

우선 실제 모델의 한국어 품질·비용·지연, 실기기 UX·성능과 운영 서명/호스팅을 확인한다. 계정·관리·결제·스토리는 공급자·권한·상품·콘텐츠 정책을 정한 뒤 요구사항·API·ERD를 갱신해 별도 TDD 단위로 진행한다. 현재 성능 목표 p95 500ms·GPT 첫 상태 1초를 측정 결과로 제시하지 않는다.

이전 전체 기획 추적표와 화면 사진은 docs/planning/implementation-status-20261106.md 및 기존 9쪽 보고서를 참고한다. 본 문서는 소스 구조·실행 흐름·구현 근거를 설명하기 위해 추가한 기술 문서다.

평가 근거 소스

taro-back / api/domain/ai_jobs.py:89-90

taro-back / api/adapters/journal_insights_postgres.py:73-75

taro / src/features/p1/P1Api.ts:36-36